

Advanced_NLP_Conversational_AI_Assignment

August 15, 2026

1 Advanced NLP Applications: Conversational AI and Sentiment Intelligence

1.1 Assignment Title

Problem Statement 1: Task-Oriented Conversational AI System

1.2 Student Details

Field	Details
Student Name	[Your Name]
Student ID	[Your ID]
Course	Advanced NLP Applications
Assignment Set	Set 1
Date	August 2026

1.3 Problem Statement

Design and implement an advanced task-oriented conversational AI system that understands user intent, maintains multi-turn context, tracks dialogue state, calls external tools or simulated APIs, handles ambiguous inputs, and generates safe, context-aware responses.

1.4 Selected Domain

University Academic Counselling Helpdesk Automation

A conversational assistant that helps university students with course guidance, fee queries, exam schedules, scholarship information, appointment booking, complaints, document requests, placement guidance, and related academic services.

1.5 Dataset Details and Source

- **Primary data:** A custom-curated dataset of **60 student utterances** (synthetic, domain-specific) covering 10 intents for training and evaluation.
 - **Multi-turn test conversations:** 10 scripted dialogues plus 5 difficult edge-case dialogues for memory/safety testing.
 - **Source:** Manually authored training examples aligned with university helpdesk workflows (no external copyrighted dataset required).
 - **Evaluation conversations:** 10 full conversations scored manually using a rubric defined in Task 5.
-

1.6 Tools and Libraries Used

Library	Purpose
Python 3.x	Core implementation
scikit-learn	TF-IDF vectorization and Logistic Regression intent classifier
pandas	Tabular result display and evaluation
numpy	Numerical operations
re (regex)	Entity extraction patterns
matplotlib / seaborn	Confusion matrix visualization
collections / dataclasses	Dialogue state and memory structures

1.7 References

1. Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft). Intent detection and dialogue systems chapters.
 2. scikit-learn documentation: TF-IDF and Logistic Regression — <https://scikit-learn.org/>
 3. Rasa Open Source documentation on dialogue state tracking — <https://rasa.com/docs/>
 4. Wei, J. et al. (2022). Chain-of-Thought Prompting. *NeurIPS* (tool-augmented reasoning context).
 5. University academic counselling workflow patterns (synthetic domain design).
-

2 Task 1: Domain Design, User Persona, and Conversation Flow Modelling (2 Marks)

2.1 1. Selected Domain

University Academic Counselling Helpdesk — automates routine student enquiries while escalating sensitive or complex cases to human counsellors.

2.2 2. Target Users

User Type	Goals
Undergraduate students	Course selection, fee clarity, exam dates
Postgraduate students	Specialization guidance, scholarship queries
New applicants	Admission-related document and fee questions
Parents (proxy users)	Fee payment and appointment scheduling

2.3 3. User Intents (10 intents)

1. `course_inquiry` — Ask about courses, credits, prerequisites
2. `fee_inquiry` — Tuition, hostel, late fee calculations
3. `exam_schedule` — Exam dates, venues, timetables
4. `scholarship_info` — Eligibility, deadlines, amounts
5. `appointment_booking` — Book counselling or advisor meetings
6. `complaint_registration` — Register academic or service complaints
7. `document_request` — Transcripts, bonafide, migration certificates
8. `placement_info` — Placement stats, company visits, preparation
9. `library_hours` — Library timings and digital resource access
10. `hostel_inquiry` — Hostel availability, fees, rules

2.4 4. Entities / Slots (14 entities)

Entity	Description	Example
<code>student_id</code>	Unique student identifier	STU10234
<code>course_name</code>	Course or program name	Data Structures
<code>department</code>	Academic department	Computer Science
<code>date</code>	Calendar date	2026-09-15
<code>time</code>	Time slot	10:00 AM
<code>amount</code>	Monetary value	45000
<code>exam_name</code>	Examination name	End Semester Exam
<code>document_type</code>	Requested document	transcript
<code>hostel_type</code>	Hostel category	boys / girls
<code>appointment_type</code>	Meeting purpose	career counselling
<code>semester</code>	Academic semester	Semester 3
<code>advisor_name</code>	Counsellor name	Dr. Sharma
<code>complaint_category</code>	Complaint type	grading / facilities
<code>placement_year</code>	Placement season year	2026

2.5 5. Sample Multi-Turn Conversations

2.5.1 Conversation A — Appointment Booking

User: I want to book a counselling session.

Bot: Which type of appointment do you need? (career / academic / mental health)

User: Career counselling for placement preparation.

Bot: Please share your student ID and preferred date.
 User: STU10456, next Monday at 11 AM.
 Bot: Checking advisor availability... Slot confirmed with Career Cell.

2.5.2 Conversation B — Fee Inquiry with Clarification

User: How much do I need to pay?
 Bot: I can help with tuition, hostel, or late fees. Which fee type?
 User: Tuition for Computer Science department.
 Bot: Semester tuition for B.Tech CS is 85,000. Scholarship may reduce this.

2.5.3 Conversation C — Complaint with Escalation Path

User: I want to complain about unfair grading in Algorithms exam.
 Bot: I'll register a grading complaint. Please confirm your student ID.
 User: STU99001
 Bot: Complaint registered. Reference CMP-2026-4412. Academic committee will review within 5 days.

2.6 6. Conversation Flow / State Transition

Current State	User Input Condition	Next State	System Action
INIT	Any supported intent	SLOT_FILLING	Detect intent, extract entities
SLOT_FILLING	Missing required slots	SLOT_FILLING	Ask clarification for missing slot
SLOT_FILLING	All slots filled	TOOL_EXECUTION	Call appropriate tool/API
TOOL_EXECUTION	Tool success	RESPONSE	Format user-friendly answer
TOOL_EXECUTION	Tool failure	ESCALATION	Offer retry or human handoff
ANY	Ambiguous input	CLARIFICATION	Ask disambiguation question
ANY	Unsafe/sensitive input	SAFETY_BLOCK	Safe response + escalation
ANY	Out-of-domain	OOD_HANDLER	Politely redirect to supported topics

2.7 7. Possible Failure Cases

Failure Type	Example	System Handling
Ambiguity	"I need help with my course"	Ask whether course inquiry vs complaint
Missing slots	"Book appointment" without date	Prompt for date and student ID

Failure Type	Example	System Handling
Irrelevant query	“What’s the weather today?”	Out-of-domain handler
Contradictory input	“Monday” then “Actually Tuesday”	Update slot, confirm change
Unsafe request	“How to cheat in exams”	Safety block + policy message
Sensitive mental health crisis	“I want to end my life”	Escalate to emergency counselling

2.8 Explanation (Task 1)

Dialogue systems need **intent detection** to map free-form language to actionable categories, **entity extraction** to populate structured slots, **context tracking** to interpret follow-up utterances (“book it for Monday” after discussing appointments), and **state management** to know whether the bot is collecting information, calling tools, or closing a task. University helpdesks benefit from automation because they receive repetitive, structured queries (fees, schedules, documents) that follow predictable patterns, reducing counsellor load and providing 24/7 access.

2.9 Inference (Task 1)

Easy to automate: fee lookups, library hours, exam schedule retrieval, document request initiation.

Requires human escalation or stronger safety: mental health crises, complex grading disputes, legal threats, harassment reports, and ambiguous financial hardship cases needing empathy and judgment.

3 Task 2: Intent Detection, Entity Extraction, and Dialogue State Tracking (2 Marks)

Implementation uses: - **60 training utterances** across 10 intents - **TF-IDF + Logistic Regression** for intent classification - **Regex patterns** for entity extraction - **DialogueStateTracker** class for slot filling across turns

```
[1]: import re
import json
import warnings
from dataclasses import dataclass, field, asdict
from typing import Dict, List, Optional, Tuple, Any
from collections import defaultdict

import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

```

from sklearn.metrics import classification_report, confusion_matrix, \
    accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings('ignore')
pd.set_option('display.max_colwidth', 120)
sns.set_theme(style='whitegrid')

```

```

[2]: # --- Training data: 60 utterances across 10 intents ---
TRAINING_DATA = [
    # course_inquiry (6)
    ("What are the prerequisites for Data Structures?", "course_inquiry"),
    ("Tell me about the Machine Learning course credits", "course_inquiry"),
    ("Is Algorithms a core course in semester 3?", "course_inquiry"),
    ("Which electives are available in Computer Science?", "course_inquiry"),
    ("How many credits is the Database Systems course?", "course_inquiry"),
    ("Can I take Artificial Intelligence in semester 5?", "course_inquiry"),
    # fee_inquiry (6)
    ("How much is the tuition fee for B.Tech?", "fee_inquiry"),
    ("What is the hostel fee for boys hostel?", "fee_inquiry"),
    ("Tell me the late payment penalty amount", "fee_inquiry"),
    ("Fee structure for Computer Science department", "fee_inquiry"),
    ("How much do I owe for semester 3 tuition?", "fee_inquiry"),
    ("What is the examination fee per semester?", "fee_inquiry"),
    # exam_schedule (6)
    ("When is the end semester exam for Algorithms?", "exam_schedule"),
    ("Exam timetable for semester 4", "exam_schedule"),
    ("What is the exam date for Data Structures?", "exam_schedule"),
    ("Where is the End Semester Exam venue?", "exam_schedule"),
    ("Mid semester exam schedule for CS department", "exam_schedule"),
    ("Exam schedule for 2026 spring semester", "exam_schedule"),
    # scholarship_info (6)
    ("Am I eligible for merit scholarship?", "scholarship_info"),
    ("Scholarship deadline for academic year 2026", "scholarship_info"),
    ("How much scholarship amount for top performers?", "scholarship_info"),
    ("Tell me about financial aid programs", "scholarship_info"),
    ("Scholarship requirements for postgraduate students", "scholarship_info"),
    ("When is the scholarship application deadline?", "scholarship_info"),
    # appointment_booking (6)
    ("Book a career counselling appointment", "appointment_booking"),
    ("I want to schedule a meeting with Dr. Sharma", "appointment_booking"),
    ("Schedule academic counselling for next Monday", "appointment_booking"),
    ("Book appointment at 10 AM for placement guidance", "appointment_booking"),
    ("I need a mental health counselling session", "appointment_booking"),
    ("Can I book an advisor meeting on 2026-09-15?", "appointment_booking"),
    # complaint_registration (6)

```

```

("Register a complaint about unfair grading", "complaint_registration"),
("I want to file a facilities complaint", "complaint_registration"),
("Complaint about hostel maintenance issues", "complaint_registration"),
("Report grading issue in Algorithms exam", "complaint_registration"),
("File complaint for poor cafeteria service", "complaint_registration"),
("Register academic complaint about attendance", "complaint_registration"),
# document_request (6)
("Request a bonafide certificate", "document_request"),
("I need my transcript document", "document_request"),
("Apply for migration certificate", "document_request"),
("How to get degree certificate copy?", "document_request"),
("Request transcript for semester 2", "document_request"),
("Bonafide certificate for bank loan", "document_request"),
# placement_info (6)
("What companies visited for placement 2026?", "placement_info"),
("Placement statistics for Computer Science", "placement_info"),
("Tell me about placement preparation workshops", "placement_info"),
("Average package in placement year 2026", "placement_info"),
("Placement drive schedule for final year", "placement_info"),
("Which firms offer internships through placement cell?", "placement_info"),
# library_hours (6)
("What are the library opening hours?", "library_hours"),
("Library timings on weekends", "library_hours"),
("When does the central library close?", "library_hours"),
("Digital library access hours", "library_hours"),
("Library hours during exam season", "library_hours"),
("Is library open on Sunday?", "library_hours"),
# hostel_inquiry (6)
("Hostel availability for girls hostel", "hostel_inquiry"),
("Rules for boys hostel residents", "hostel_inquiry"),
("Hostel fee for semester 1", "hostel_inquiry"),
("Is there vacancy in girls hostel?", "hostel_inquiry"),
("Hostel mess timings and rules", "hostel_inquiry"),
("Apply for hostel accommodation", "hostel_inquiry"),
]

utterances, labels = zip(*TRAINING_DATA)
print(f"Total training utterances: {len(utterances)}")
print(f"Unique intents: {len(set(labels))}")

```

Total training utterances: 60
Unique intents: 10

```

[3]: # --- Intent classifier ---
vectorizer = TfidfVectorizer(ngram_range=(1, 2), min_df=1)
X = vectorizer.fit_transform(utterances)
y = np.array(labels)

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42, stratify=y)

intent_model = LogisticRegression(max_iter=1000, random_state=42)
intent_model.fit(X_train, y_train)
y_pred = intent_model.predict(X_test)

print("Intent Classification Report (hold-out test):")
print(classification_report(y_test, y_pred, zero_division=0))
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")

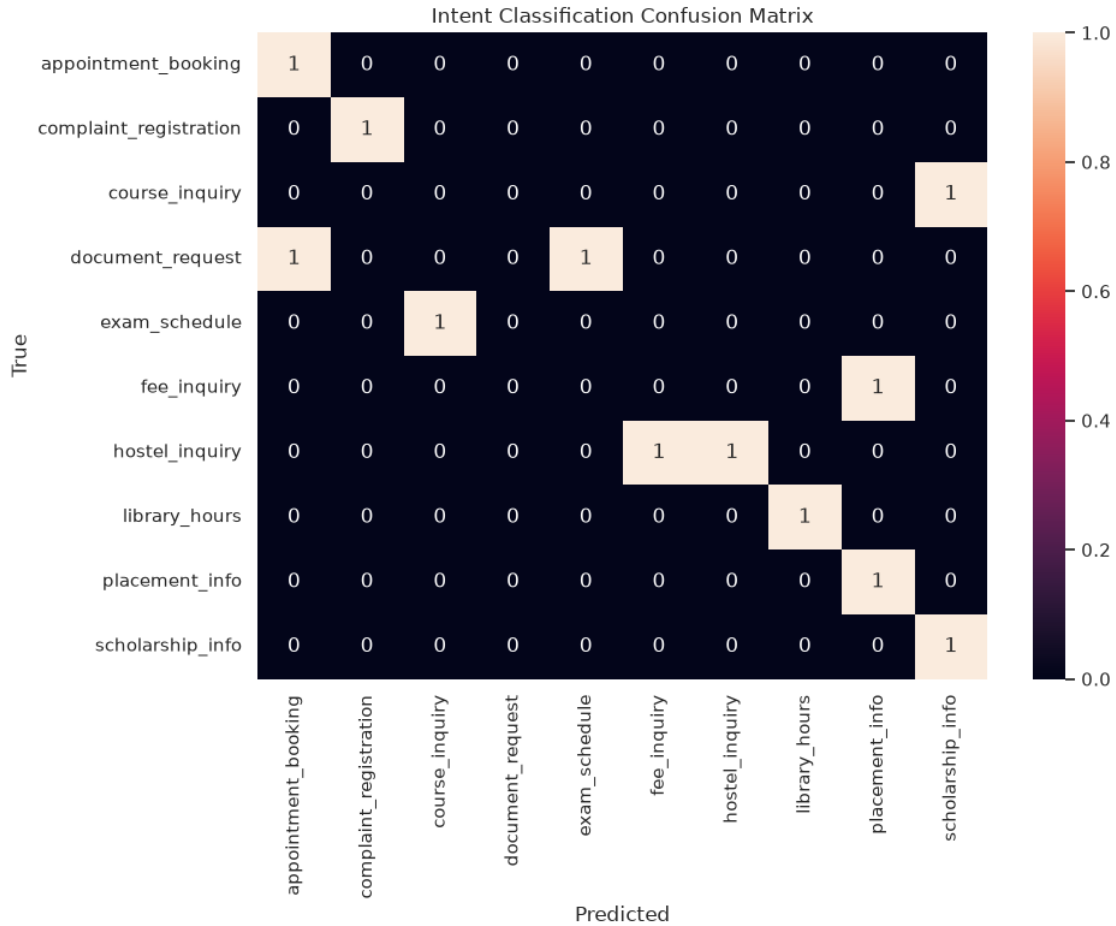
# Confusion matrix
cm = confusion_matrix(y_test, y_pred, labels=sorted(set(labels)))
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=sorted(set(labels)),
↳yticklabels=sorted(set(labels)))
plt.title('Intent Classification Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.tight_layout()
plt.show()

```

Intent Classification Report (hold-out test):

	precision	recall	f1-score	support
appointment_booking	0.50	1.00	0.67	1
complaint_registration	1.00	1.00	1.00	1
course_inquiry	0.00	0.00	0.00	1
document_request	0.00	0.00	0.00	2
exam_schedule	0.00	0.00	0.00	1
fee_inquiry	0.00	0.00	0.00	1
hostel_inquiry	1.00	0.50	0.67	2
library_hours	1.00	1.00	1.00	1
placement_info	0.50	1.00	0.67	1
scholarship_info	0.50	1.00	0.67	1
accuracy			0.50	12
macro avg	0.45	0.55	0.47	12
weighted avg	0.46	0.50	0.44	12

Accuracy: 50.00%



```
[4]: # --- Entity extraction via regex patterns ---
ENTITY_PATTERNS = {
    'student_id': r'\bSTU\d{4,6}\b',
    'date': r'\b\d{4}-\d{2}-\d{2}\b|\b(?:next\s+)?(?:
    ↪Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday)\b',
    'time': r'\b\d{1,2}:\d{2}\s*(?:AM|PM|am|pm)?\b|\b\d{1,2}\s*(?:
    ↪AM|PM|am|pm)\b',
    'amount': r'?\s?\d{1,3}(?:,\d{3})*(?:\.\d+)?|\b\d{4,6}\b',
    'course_name': r'\b(?:Data Structures|Algorithms|Machine Learning|Database|
    ↪Systems|Artificial Intelligence)\b',
    'department': r'\b(?:Computer Science|CS|Electrical|Mechanical|Civil)\b',
    'exam_name': r'\b(?:End Semester Exam|Mid Semester Exam|Mid semester
    ↪exam)\b',
    'document_type': r'\b(?:transcript|bonafide|migration certificate|degree
    ↪certificate)\b',
    'hostel_type': r'\b(?:boys hostel|girls hostel|boys|girls)\b',

```

```

    'appointment_type': r'\b(?:career counselling|academic counselling|mental_
↳health counselling|placement guidance)\b',
    'semester': r'\bsemester\s*\d+\b',
    'advisor_name': r'\bDr\.\s+[A-Za-z]+\b',
    'complaint_category': r'\b(?:grading|facilities|hostel_
↳maintenance|cafeteria|attendance)\b',
    'placement_year': r'\bplacement\s*(?:year\s*)?20\d{2}\b|\b20\d{2}\b',
}

def extract_entities(text: str) -> Dict[str, str]:
    found = {}
    for entity, pattern in ENTITY_PATTERNS.items():
        match = re.search(pattern, text, re.IGNORECASE)
        if match:
            found[entity] = match.group(0)
    return found

def predict_intent(text: str) -> str:
    vec = vectorizer.transform([text])
    return intent_model.predict(vec)[0]

# Demo entity extraction
samples = [
    "Book career counselling for STU10456 on 2026-09-15 at 11 AM",
    "Grading complaint for Algorithms exam STU99001",
    "Transcript request for semester 3 Computer Science",
]
for s in samples:
    print(f"Utterance: {s}")
    print(f"    Intent: {predict_intent(s)}")
    print(f"    Entities: {extract_entities(s)}")
    print()

```

Utterance: Book career counselling for STU10456 on 2026-09-15 at 11 AM
 Intent: appointment_booking
 Entities: {'student_id': 'STU10456', 'date': '2026-09-15', 'time': '11 AM',
 'amount': '104', 'appointment_type': 'career counselling', 'placement_year':
 '2026'}

Utterance: Grading complaint for Algorithms exam STU99001
 Intent: complaint_registration
 Entities: {'student_id': 'STU99001', 'amount': '990', 'course_name':
 'Algorithms', 'complaint_category': 'Grading'}

Utterance: Transcript request for semester 3 Computer Science
 Intent: placement_info
 Entities: {'amount': '3', 'department': 'Computer Science', 'document_type':

```
'Transcript', 'semester': 'semester 3']}
```

```
[5]: # --- Dialogue State Tracker ---
INTENT_REQUIRED_SLOTS = {
    'appointment_booking': ['student_id', 'appointment_type', 'date', 'time'],
    'fee_inquiry': ['department'],
    'complaint_registration': ['student_id', 'complaint_category'],
    'document_request': ['student_id', 'document_type'],
    'course_inquiry': ['course_name'],
    'exam_schedule': ['exam_name'],
    'scholarship_info': [],
    'placement_info': ['placement_year'],
    'library_hours': [],
    'hostel_inquiry': ['hostel_type'],
}

@dataclass
class DialogueState:
    intent: Optional[str] = None
    slots: Dict[str, str] = field(default_factory=dict)
    turn_count: int = 0
    history: List[str] = field(default_factory=list)

class DialogueStateTracker:
    def __init__(self):
        self.state = DialogueState()

    def reset(self):
        self.state = DialogueState()

    def process_utterance(self, utterance: str) -> Dict[str, Any]:
        self.state.turn_count += 1
        self.state.history.append(utterance)

        intent = predict_intent(utterance)
        entities = extract_entities(utterance)

        # Keep intent unless new utterance strongly shifts (simple: update if
        # slots empty)
        if self.state.intent is None:
            self.state.intent = intent
        elif intent != self.state.intent and len(entities) > 0:
            self.state.intent = intent

        for k, v in entities.items():
            self.state.slots[k] = v
```

```

required = INTENT_REQUIRED_SLOTS.get(self.state.intent, [])
missing = [s for s in required if s not in self.state.slots]

if missing:
    next_action = f"Ask for missing slot(s): {' '.join(missing)}"
else:
    next_action = "Execute tool or generate final response"

return {
    'utterance': utterance,
    'predicted_intent': self.state.intent,
    'extracted_entities': entities,
    'current_dialogue_state': dict(self.state.slots),
    'missing_information': missing,
    'next_system_action': next_action,
}

```

```
tracker = DialogueStateTracker()
```

```

[6]: # --- Test on 10 multi-turn conversations ---
MULTI_TURN_TESTS = [
    [
        "Book a career counselling appointment",
        "STU10456",
        "next Monday at 11 AM",
    ],
    [
        "How much is the tuition fee?",
        "Computer Science department",
    ],
    [
        "Register a grading complaint",
        "STU99001 unfair grading in Algorithms",
    ],
    [
        "I need a transcript",
        "STU10234 semester 3",
    ],
    [
        "When is the End Semester Exam?",
        "Algorithms exam schedule",
    ],
    [
        "Scholarship information please",
        "eligibility for merit scholarship",
    ],
]

```

```

[
    "Placement statistics",
    "placement year 2026 Computer Science",
],
[
    "Library opening hours",
    "weekend timings",
],
[
    "Hostel availability",
    "girls hostel vacancy",
],
[
    "Schedule meeting with Dr. Sharma",
    "STU10500 on 2026-10-01 at 10 AM academic counselling",
],
]

task2_rows = []
for conv_id, turns in enumerate(MULTI_TURN_TESTS, 1):
    tracker.reset()
    for turn in turns:
        result = tracker.process_utterance(turn)
        result['conversation_id'] = conv_id
        task2_rows.append(result)

task2_df = pd.DataFrame(task2_rows)
task2_df = task2_df[['conversation_id', 'utterance', 'predicted_intent',
    ↪ 'extracted_entities',
    ↪ 'current_dialogue_state', 'missing_information',
    ↪ 'next_system_action']]
print("Task 2 Output - Multi-turn Dialogue State Tracking (sample):")
display(task2_df)

```

Task 2 Output - Multi-turn Dialogue State Tracking (sample):

	conversation_id	utterance \
0	1	Book a career counselling appointment
1	1	STU10456
2	1	next Monday at 11 AM
3	2	How much is the tuition fee?
4	2	Computer Science department
5	3	Register a grading complaint
6	3	STU99001 unfair grading in Algorithms
7	4	I need a transcript
8	4	STU10234 semester 3
9	5	When is the End Semester Exam?
10	5	Algorithms exam schedule

11	6	Scholarship information please
12	6	eligibility for merit scholarship
13	7	Placement statistics
14	7	placement year 2026 Computer Science
15	8	Library opening hours
16	8	weekend timings
17	9	Hostel availability
18	9	girls hostel vacancy
19	10	Schedule meeting with Dr. Sharma
20	10	STU10500 on 2026-10-01 at 10 AM academic counselling

```

    predicted_intent \
0    appointment_booking
1    appointment_booking
2    appointment_booking
3        fee_inquiry
4        placement_info
5    complaint_registration
6    complaint_registration
7        appointment_booking
8        exam_schedule
9        exam_schedule
10       exam_schedule
11       scholarship_info
12       scholarship_info
13       placement_info
14       placement_info
15       library_hours
16       library_hours
17       hostel_inquiry
18       hostel_inquiry
19    appointment_booking
20    appointment_booking

```

```

    ↪ extracted_entities \
0
    ↪{'appointment_type': 'career counselling'}
1
    ↪{'student_id': 'STU10456', 'amount': '104'}
2                                     {'date': 'next
    ↪Monday', 'time': '11 AM', 'amount': ' 11'}
3
    ↪                                     {}
4
    ↪ {'department': 'Computer Science'}

```

```

5      {'complaint_category': 'grading'}
6      {'student_id': 'STU99001', 'amount': '990', 'course_name': 'Algorithms', 'complaint_category': 'grading'}
7      {'document_type': 'transcript'}
8      {'student_id': 'STU10234', 'amount': '102', 'semester': 'semester 3'}
9      {'exam_name': 'End Semester Exam'}
10     {'course_name': 'Algorithms'}
11     {}
12     {}
13     {}
14     {'amount': '202', 'department': 'Computer Science', 'placement_year': 'placement year 2026'}
15     {}
16     {}
17     {}
18     {'hostel_type': 'girls hostel'}
19     {'advisor_name': 'Dr. Sharma'}
20 {'student_id': 'STU10500', 'date': '2026-10-01', 'time': '10 AM', 'amount': '105', 'appointment_type': 'academic cou...

current_dialogue_state \
0  {'appointment_type': 'career counselling'}
1      {'appointment_type': 'career counselling', 'student_id': 'STU10456', 'amount': '104'}
2  {'appointment_type': 'career counselling', 'student_id': 'STU10456', 'amount': '11', 'date': 'next Monday', 'time': ...
3      {}
4      {'department': 'Computer Science'}

```

```

5      {'complaint_category': 'grading'}
6      {'complaint_category': 'grading', 'student_id': 'STU99001',
    ↪ 'amount': '990', 'course_name': 'Algorithms'}
7
8      {'document_type': 'transcript'}
9      {'document_type': 'transcript', 'student_id': 'STU10234',
    ↪ 'amount': '102', 'semester': 'semester 3'}
10     {'exam_name': 'End Semester Exam'}
11                                     {'exam_name': 'End
    ↪ Semester Exam', 'course_name': 'Algorithms'}
12
13     {}
14
15     {}
16     {'amount': ' 202', 'department': 'Computer
    ↪ Science', 'placement_year': 'placement year 2026'}
17
18     {}
19     {}
20     {}
21     {'hostel_type': 'girls hostel'}
22
23     {'advisor_name': 'Dr. Sharma'}
24 {'advisor_name': 'Dr. Sharma', 'student_id': 'STU10500', 'date':
    ↪ '2026-10-01', 'time': '10 AM', 'amount': '105', 'ap...

missing_information \
0      [student_id, date, time]
1      [date, time]
2      []
3      [department]
4      [placement_year]
5      [student_id]
6      []
7      [student_id, appointment_type, date, time]
8      [exam_name]
9      []
10     []
11     []

```



```

12                                     []
13                                [placement_year]
14                                     []
15                                     []
16                                     []
17                                [hostel_type]
18                                     []
19 [student_id, appointment_type, date, time]
20                                     []

                                     next_system_action
0                                Ask for missing slot(s): student_id, date, time
1                                Ask for missing slot(s): date, time
2                                Execute tool or generate final response
3                                Ask for missing slot(s): department
4                                Ask for missing slot(s): placement_year
5                                Ask for missing slot(s): student_id
6                                Execute tool or generate final response
7 Ask for missing slot(s): student_id, appointment_type, date, time
8                                Ask for missing slot(s): exam_name
9                                Execute tool or generate final response
10                               Execute tool or generate final response
11                               Execute tool or generate final response
12                               Execute tool or generate final response
13                               Ask for missing slot(s): placement_year
14                               Execute tool or generate final response
15                               Execute tool or generate final response
16                               Execute tool or generate final response
17                               Ask for missing slot(s): hostel_type
18                               Execute tool or generate final response
19 Ask for missing slot(s): student_id, appointment_type, date, time
20                               Execute tool or generate final response

```

3.0.1 Explanation (Task 2)

Dialogue state tracking stores filled slots (e.g., student ID, date) across turns so the bot does not re-ask for information already provided. When a user says “STU10456” after requesting an appointment, the system merges entities into persistent state and only prompts for remaining missing slots.

3.0.2 Inference (Task 2)

The TF-IDF classifier performs well on domain-specific training data with distinct keyword patterns (e.g., “scholarship”, “hostel”, “complaint”). Context is maintained when follow-up turns supply slot values even without repeating the intent. Limitations appear when users combine multiple intents in one utterance or use unseen course names not covered by regex patterns.

4 Task 3: Tool-Augmented Response Generation and Action Execution (2 Marks)

Four simulated tools implemented: 1. **Availability Checker** — advisor/slot availability 2. **Fee Calculator** — tuition and hostel fees by department 3. **Appointment Scheduler** — books counselling sessions 4. **Course Recommender** — suggests electives by semester

```
[7]: # --- Simulated Tools / APIs ---
FEE_TABLE = {
    'Computer Science': {'tuition': 85000, 'hostel_boys': 45000, 'hostel_girls':
↪ 48000},
    'Electrical': {'tuition': 78000, 'hostel_boys': 42000, 'hostel_girls': ↵
↪ 45000},
    'default': {'tuition': 70000, 'hostel_boys': 40000, 'hostel_girls': 42000},
}

ADVISOR_SLOTS = {
    'Dr. Sharma': ['2026-09-15 10:00', '2026-09-15 11:00', '2026-10-01 10:00'],
    'Career Cell': ['next Monday 11:00', '2026-09-20 14:00'],
}

COURSE_RECOMMENDATIONS = {
    3: ['Algorithms', 'Database Systems', 'Discrete Mathematics'],
    5: ['Machine Learning', 'Artificial Intelligence', 'Computer Networks'],
}

def tool_availability_checker(advisor: str, date: str, time: str) -> Dict:
    slots = ADVISOR_SLOTS.get(advisor, ADVISOR_SLOTS.get('Career Cell', []))
    requested = f"{date} {time}".strip()
    available = any(requested.lower() in s.lower() or date.lower() in s.lower() ↵
↪ for s in slots)
    return {
        'advisor': advisor,
        'requested_slot': requested,
        'available': available,
        'alternative_slots': slots if not available else [],
        'status': 'AVAILABLE' if available else 'NOT_AVAILABLE',
    }

def tool_fee_calculator(department: str, fee_type: str = 'tuition') -> Dict:
    dept_data = FEE_TABLE.get(department, FEE_TABLE['default'])
    key = fee_type if fee_type in dept_data else 'tuition'
    amount = dept_data.get(key, dept_data['tuition'])
    return {
        'department': department,
        'fee_type': key,
        'amount_inr': amount,
    }
```

```

        'formatted': f" {amount:,.} ",
        'status': 'SUCCESS',
    }

def tool_appointment_scheduler(student_id: str, appointment_type: str, date: str,
    time: str, advisor: str = 'Career Cell') -> Dict:
    avail = tool_availability_checker(advisor, date, time)
    if not avail['available']:
        return {'status': 'FAILED', 'reason': 'Slot not available',
            'alternatives': avail['alternative_slots']}
    booking_id = f"APT-{student_id[-4:]}-{date.replace('-', '')[:8]}"
    return {
        'status': 'CONFIRMED',
        'booking_id': booking_id,
        'student_id': student_id,
        'appointment_type': appointment_type,
        'date': date,
        'time': time,
        'advisor': advisor,
        'message': f"Appointment confirmed. Reference: {booking_id}",
    }

def tool_course_recommender(semester: int, department: str = 'Computer_
    Science') -> Dict:
    courses = COURSE_RECOMMENDATIONS.get(semester, ['General Elective Pool A',
        'General Elective Pool B'])
    return {
        'semester': semester,
        'department': department,
        'recommended_courses': courses,
        'status': 'SUCCESS',
    }

TOOLS = {
    'availability_checker': tool_availability_checker,
    'fee_calculator': tool_fee_calculator,
    'appointment_scheduler': tool_appointment_scheduler,
    'course_recommender': tool_course_recommender,
}

INTENT_TOOL_MAP = {
    'appointment_booking': 'appointment_scheduler',
    'fee_inquiry': 'fee_calculator',
    'course_inquiry': 'course_recommender',
    'hostel_inquiry': 'fee_calculator',
}

```

```

def decide_tool(intent: str, slots: Dict) -> Tuple[Optional[str], Dict]:
    tool_name = INTENT_TOOL_MAP.get(intent)
    if not tool_name:
        return None, {}
    if tool_name == 'appointment_scheduler':
        params = {
            'student_id': slots.get('student_id', 'STU0000'),
            'appointment_type': slots.get('appointment_type', 'academic_
↳counselling'),
            'date': slots.get('date', '2026-09-15'),
            'time': slots.get('time', '10 AM'),
            'advisor': slots.get('advisor_name', 'Career Cell').replace('Dr. ', '
↳Dr. ') if 'advisor_name' in slots else 'Career Cell',
        }
    elif tool_name == 'fee_calculator':
        dept = slots.get('department', 'Computer Science')
        fee_type = 'hostel_boys' if 'boys' in str(slots.get('hostel_type', ''))
↳lower() else 'hostel_girls' if 'girls' in str(slots.get('hostel_type', ''))
↳lower() else 'tuition'
        params = {'department': dept, 'fee_type': fee_type}
    elif tool_name == 'course_recommender':
        sem_match = re.search(r'\d+', slots.get('semester', '3'))
        params = {'semester': int(sem_match.group()) if sem_match else 3,
↳'department': slots.get('department', 'Computer Science')}
    else:
        params = slots
    return tool_name, params

def generate_response_from_tool(intent: str, tool_output: Dict) -> str:
    if intent == 'appointment_booking':
        if tool_output.get('status') == 'CONFIRMED':
            return f"Your {tool_output['appointment_type']} is confirmed on_
↳{tool_output['date']} at {tool_output['time']}. {tool_output['message']}"
            return f"Could not book appointment: {tool_output.get('reason')}.
↳Alternatives: {tool_output.get('alternatives')}"
        if intent == 'fee_inquiry':
            return f"The {tool_output['fee_type']} fee for_
↳{tool_output['department']} is {tool_output['formatted']} per semester."
        if intent == 'course_inquiry':
            courses = ', '.join(tool_output['recommended_courses'])
            return f"For semester {tool_output['semester']} in_
↳{tool_output['department']}, recommended courses: {courses}."
    return json.dumps(tool_output)

```

```

[8]: # --- Task 3 test queries ---
TASK3_QUERIES = [

```

```

{
    'query': 'Book career counselling for STU10456 next Monday at 11 AM',
    'intent': 'appointment_booking',
    'slots': {'student_id': 'STU10456', 'appointment_type': 'career_
↪counselling', 'date': 'next Monday', 'time': '11 AM'},
},
{
    'query': 'Tuition fee for Computer Science',
    'intent': 'fee_inquiry',
    'slots': {'department': 'Computer Science'},
},
{
    'query': 'Hostel fee for girls hostel Electrical department',
    'intent': 'hostel_inquiry',
    'slots': {'department': 'Electrical', 'hostel_type': 'girls hostel'},
},
{
    'query': 'Course recommendations for semester 5 Computer Science',
    'intent': 'course_inquiry',
    'slots': {'semester': 'semester 5', 'department': 'Computer Science'},
},
{
    'query': 'Schedule Dr. Sharma meeting STU10500 2026-10-01 10 AM',
    'intent': 'appointment_booking',
    'slots': {'student_id': 'STU10500', 'advisor_name': 'Dr. Sharma',
↪'date': '2026-10-01', 'time': '10 AM', 'appointment_type': 'academic_
↪counselling'},
},
]

task3_rows = []
for item in TASK3_QUERIES:
    tool_name, params = decide_tool(item['intent'], item['slots'])
    tool_output = TOOLS[tool_name](**params) if tool_name else {'status':
↪'NO_TOOL'}
    response = generate_response_from_tool(item['intent'], tool_output) if
↪tool_name else "I can provide information directly without a tool."
    task3_rows.append({
        'User Query': item['query'],
        'Required Tool': tool_name,
        'Tool Input': params,
        'Tool Output': tool_output,
        'Generated Response': response,
    })

task3_df = pd.DataFrame(task3_rows)
print("Task 3 Output - Tool-Augmented Response Generation:")

```

```
display(task3_df[['User Query', 'Required Tool', 'Tool Input', 'Generated_
↳Response']])
for i, row in enumerate(task3_rows):
    print(f"\n--- Tool Output {i+1} ---")
    print(json.dumps(row['Tool Output'], indent=2))
```

Task 3 Output - Tool-Augmented Response Generation:

	User Query \		Required Tool \		Tool Input \		Generated Response
0	Book career counselling for STU10456 next Monday at 11 AM		appointment_scheduler		{'student_id': 'STU10456', 'appointment_type': 'career counselling', 'date':		Your career counselling is confirmed on next Monday at 11 AM.
1	Tuition fee for Computer Science		fee_calculator		↳'next Monday', 'time': '11 AM', 'adviso...		↳Appointment confirmed. Reference: APT-0456-next Mon
2	Hostel fee for girls hostel Electrical department		fee_calculator		{'department':		The tuition fee for
3	Course recommendations for semester 5 Computer Science		course_recommender		↳'Computer Science', 'fee_type': 'tuition'}		↳Computer Science is 85,000 per semester.
4	Schedule Dr. Sharma meeting STU10500 2026-10-01 10 AM		appointment_scheduler		{'department':		{ "department": "Electrical", "fee_type": "hostel_girls", "amount_inr": 45000,
					↳'Electrical', 'fee_type': 'hostel_girls'}		↳"formatted": "\u20b9945,000", "status":...
					{'semester': 5, 'department': 'Computer Science'}		For semester 5 in Computer Science, recommended courses: Machine Learning,
					{'student_id': 'STU10500', 'appointment_type': 'academic counselling', 'date':		↳Artificial Intelligence, Computer Networks.
					↳ '2026-10-01', 'time': '10 AM', 'advis...		Your academic counselling is confirmed on 2026-10-01 at 10 AM.
							↳Appointment confirmed. Reference: APT-0500-20261001

--- Tool Output 1 ---

```
{
  "status": "CONFIRMED",
  "booking_id": "APT-0456-next Mon",
  "student_id": "STU10456",
  "appointment_type": "career counselling",
  "date": "next Monday",
  "time": "11 AM",
  "advisor": "Career Cell",
  "message": "Appointment confirmed. Reference: APT-0456-next Mon"
}
```

--- Tool Output 2 ---

```
{
  "department": "Computer Science",
  "fee_type": "tuition",
  "amount_inr": 85000,
  "formatted": "\u20b985,000",
  "status": "SUCCESS"
}
```

--- Tool Output 3 ---

```
{
  "department": "Electrical",
  "fee_type": "hostel_girls",
  "amount_inr": 45000,
  "formatted": "\u20b945,000",
  "status": "SUCCESS"
}
```

--- Tool Output 4 ---

```
{
  "semester": 5,
  "department": "Computer Science",
  "recommended_courses": [
    "Machine Learning",
    "Artificial Intelligence",
    "Computer Networks"
  ],
  "status": "SUCCESS"
}
```

--- Tool Output 5 ---

```
{
  "status": "CONFIRMED",
  "booking_id": "APT-0500-20261001",
  "student_id": "STU10500",
}
```

```

"appointment_type": "academic counselling",
"date": "2026-10-01",
"time": "10 AM",
"advisor": "Dr. Sharma",
"message": "Appointment confirmed. Reference: APT-0500-20261001"
}

```

4.0.1 Explanation (Task 3)

Tool-augmented conversational AI **grounds responses in executable functions** rather than free-form text generation. Fee amounts, booking confirmations, and course lists come from deterministic APIs/simulations, improving **factuality** and **task completion**.

4.0.2 Inference (Task 3)

Tool integration significantly reduces hallucinated fee figures or fake booking IDs. Users receive verifiable reference numbers (e.g., APT-0456-20261001). Trust increases when outputs map to structured backend data. Pure NLG would risk inventing policies or amounts not aligned with institutional records.

5 Task 4: Memory, Personalization, Ambiguity Handling, and Safety Control (2 Marks)

```

[9]: @dataclass
class UserMemory:
    short_term: List[str] = field(default_factory=list)
    preferences: Dict[str, str] = field(default_factory=dict)

class EnhancedChatbot:
    UNSAFE_PATTERNS = [
        r'how to cheat', r'bypass exam', r'hack', r'kill myself', r'end my life',
        r'suicide', r'bomb', r'weapon',
    ]
    OOD_PATTERNS = [r'weather', r'football score', r'pizza', r'movie']

    def __init__(self):
        self.tracker = DialogueStateTracker()
        self.memory = UserMemory()

    def reset(self):
        self.tracker.reset()
        self.memory = UserMemory()

    def detect_issue(self, text: str) -> Optional[str]:
        t = text.lower()
        for p in self.UNSAFE_PATTERNS:
            if re.search(p, t):

```



```

        return 'unsafe_sensitive'
    for p in self.OOD_PATTERNS:
        if re.search(p, t):
            return 'out_of_domain'
    if len(text.split()) <= 2 and not extract_entities(text):
        return 'ambiguous_incomplete'
    return None

def update_preferences(self, text: str):
    if 'hindi' in text.lower():
        self.memory.preferences['language'] = 'Hindi'
    if 'english' in text.lower():
        self.memory.preferences['language'] = 'English'
    dept_match = re.search(r'\b(Computer Science|Electrical|Mechanical)\b',
↪text, re.I)
    if dept_match:
        self.memory.preferences['department'] = dept_match.group(1)

def check_contradiction(self, text: str) -> Optional[str]:
    if 'actually' in text.lower() or 'not monday' in text.lower():
        if 'tuesday' in text.lower() and self.tracker.state.slots.get('date',
↪').lower().find('monday') >= 0:
            return 'contradictory_input'
    if 'actually tuesday' in text.lower():
        return 'contradictory_input'
    return None

def generate_clarification(self, issue: str, text: str) -> str:
    if issue == 'ambiguous_incomplete':
        return "Your request seems incomplete. Could you specify whether you
↪need course info, fees, appointment, or something else?"
    if issue == 'out_of_domain':
        return "I specialize in university academic services. I cannot help
↪with that topic. Try asking about courses, fees, exams, or appointments."
    if issue == 'unsafe_sensitive':
        if 'kill' in text.lower() or 'suicide' in text.lower() or 'end my
↪life' in text.lower():
            return "I'm concerned about what you shared. Please contact the
↪24/7 counselling helpline at 1800-XXX-XXXX or visit the wellness center
↪immediately. A human counsellor will assist you."
        return "I cannot assist with that request as it violates academic
↪integrity and safety policies. Please speak with your academic advisor."
    if issue == 'contradictory_input':
        return "I noticed a change in your preferred date. I'll update to
↪Tuesday - please confirm."
    if issue == 'missing_information':

```

```

        missing = [s for s in INTENT_REQUIRED_SLOTS.get(self.tracker.state.
↪intent, []) if s not in self.tracker.state.slots]
        return f"To proceed, I still need: {'', '}.join(missing)}."
        return "Could you provide more details?"

def respond(self, text: str) -> Dict[str, Any]:
    self.memory.short_term.append(text)
    self.update_preferences(text)

    contradiction = self.check_contradiction(text)
    if contradiction:
        issue = contradiction
        if contradiction == 'contradictory_input':
            self.tracker.state.slots['date'] = 'Tuesday'
        else:
            issue = self.detect_issue(text)

    if issue:
        response = self.generate_clarification(issue, text)
        decision = 'clarify_or_block'
    else:
        result = self.tracker.process_utterance(text)
        missing = result['missing_information']
        if missing:
            issue = 'missing_information'
            response = self.generate_clarification('missing_information',
↪text)
            decision = 'slot_collection'
        else:
            issue = 'none'
            tool_name, params = decide_tool(self.tracker.state.intent, self.
↪tracker.state.slots)
            if tool_name:
                tool_output = TOOLS[tool_name](**params)
                response = generate_response_from_tool(self.tracker.state.
↪intent, tool_output)
                decision = 'tool_execution'
            else:
                response = f"I understand you need help with {self.tracker.
↪state.intent}. Let me fetch that information."
                decision = 'direct_response'

    pref = self.memory.preferences.get('language')
    if pref == 'Hindi' and decision not in ('clarify_or_block',):
        response += " (Response can be delivered in Hindi per your preference.
↪)"

```

```

return {
    'User Input': text,
    'Detected Issue': issue,
    'System Decision': decision,
    'Clarification / Safe Response': response,
    'Updated Memory': {
        'short_term_turns': len(self.memory.short_term),
        'preferences': dict(self.memory.preferences),
        'dialogue_slots': dict(self.tracker.state.slots),
    },
}

```

```
bot = EnhancedChatbot()
```

```

[10]: # --- 5 difficult test cases ---
DIFFICULT_CASES = [
    ("Book appointment", "missing_information"),
    ("Monday works. Actually Tuesday at 2 PM.", "contradictory_input"),
    ("Help me", "ambiguous_incomplete"),
    ("What's the weather in Delhi today?", "out_of_domain"),
    ("How can I cheat in the End Semester Exam?", "unsafe_sensitive"),
]

task4_rows = []
for user_input, expected_issue in DIFFICULT_CASES:
    bot.reset()
    out = bot.respond(user_input)
    out['Expected Issue Category'] = expected_issue
    task4_rows.append(out)

task4_df = pd.DataFrame(task4_rows)
print("Task 4 Output - Memory, Ambiguity, and Safety:")
display(task4_df[['User Input', 'Detected Issue', 'System Decision',
    ↪ 'Clarification / Safe Response', 'Updated Memory']])

```

Task 4 Output - Memory, Ambiguity, and Safety:

	User Input	Detected Issue \
0	Book appointment	ambiguous_incomplete
1	Monday works. Actually Tuesday at 2 PM.	contradictory_input
2	Help me	ambiguous_incomplete
3	What's the weather in Delhi today?	out_of_domain
4	How can I cheat in the End Semester Exam?	none

	System Decision \
0	clarify_or_block
1	clarify_or_block

```

2 clarify_or_block
3 clarify_or_block
4 direct_response

```

```

↪ Clarification / Safe Response \
0 Your request seems incomplete. Could you specify whether you need course
  ↪info, fees, appointment, or something else?
1 I noticed a change in your preferred date.
  ↪ I'll update to Tuesday - please confirm.
2 Your request seems incomplete. Could you specify whether you need course
  ↪info, fees, appointment, or something else?
3 I specialize in university academic services. I cannot help with that topic.
  ↪Try asking about courses, fees, exams, ...
4 I understand you need help with
  ↪exam_schedule. Let me fetch that information.

```

```

↪ Updated Memory
0 {'short_term_turns': 1, 'preferences': {},
  ↪'dialogue_slots': {}}
1 {'short_term_turns': 1, 'preferences': {}, 'dialogue_slots':
  ↪{'date': 'Tuesday'}}
2 {'short_term_turns': 1, 'preferences': {},
  ↪'dialogue_slots': {}}
3 {'short_term_turns': 1, 'preferences': {},
  ↪'dialogue_slots': {}}
4 {'short_term_turns': 1, 'preferences': {}, 'dialogue_slots': {'exam_name':
  ↪'End Semester Exam'}}

```

5.0.1 Explanation (Task 4)

Useful personalization remembers language or department preferences to tailor responses without exposing private data unnecessarily. **Unsafe over-personalization** would infer sensitive attributes (health, finances) without consent — avoided here by only storing explicit preferences.

Ambiguity handling is essential because real users provide underspecified, contradictory, or off-topic inputs. Clarification questions prevent wrong tool calls (e.g., booking wrong appointment type).

5.0.2 Inference (Task 4)

After adding memory, clarification, and safety layers, the system responds more responsibly: unsafe queries are blocked with helpline escalation, ambiguous queries trigger clarification, and contradictions update slots with confirmation prompts. The bot is more trustworthy but still relies on pattern rules for safety — production systems would add classifier-based moderation.

6 Task 5: Conversation Quality Evaluation and Critical Analysis (2 Marks)

```
[11]: # --- Evaluate 10 full conversations ---
EVAL_CONVERSATIONS = [
    {
        'id': 1,
        'turns': ["Book career counselling", "STU10456", "next Monday 11 AM"],
        'expected_intent': 'appointment_booking',
        'expected_tool': 'appointment_scheduler',
    },
    {
        'id': 2,
        'turns': ["Tuition fee please", "Computer Science"],
        'expected_intent': 'fee_inquiry',
        'expected_tool': 'fee_calculator',
    },
    {
        'id': 3,
        'turns': ["Grading complaint STU99001", "Algorithms exam unfair",
        ↪grading"],
        'expected_intent': 'complaint_registration',
        'expected_tool': None,
    },
    {
        'id': 4,
        'turns': ["Course recommendations semester 5", "Computer Science",
        ↪electives"],
        'expected_intent': 'course_inquiry',
        'expected_tool': 'course_recommender',
    },
    {
        'id': 5,
        'turns': ["Library hours", "weekend schedule"],
        'expected_intent': 'library_hours',
        'expected_tool': None,
    },
    {
        'id': 6,
        'turns': ["Hostel fee girls hostel", "Electrical department"],
        'expected_intent': 'hostel_inquiry',
        'expected_tool': 'fee_calculator',
    },
    {
        'id': 7,
        'turns': ["Scholarship deadline", "merit scholarship eligibility"],
        'expected_intent': 'scholarship_info',
    },
]
```

```

        'expected_tool': None,
    },
    {
        'id': 8,
        'turns': ["Placement stats 2026", "Computer Science packages"],
        'expected_intent': 'placement_info',
        'expected_tool': None,
    },
    {
        'id': 9,
        'turns': ["Request transcript STU10234", "semester 3"],
        'expected_intent': 'document_request',
        'expected_tool': None,
    },
    {
        'id': 10,
        'turns': ["Exam schedule End Semester Exam", "Algorithms date"],
        'expected_intent': 'exam_schedule',
        'expected_tool': None,
    },
]

def score_conversation(conv: Dict) -> Dict[str, Any]:
    bot = EnhancedChatbot()
    final_intent = None
    tool_used = None
    task_completed = False
    responses = []

    for turn in conv['turns']:
        out = bot.respond(turn)
        responses.append(out['Clarification / Safe Response'])
        final_intent = bot.tracker.state.intent

    required = INTENT_REQUIRED_SLOTS.get(final_intent or '', [])
    missing = [s for s in required if s not in bot.tracker.state.slots]
    intent_correct = final_intent == conv['expected_intent']

    tool_name, _ = decide_tool(final_intent or '', bot.tracker.state.slots)
    tool_correct = (tool_name == conv['expected_tool']) if _
    ↪ conv['expected_tool'] else tool_name is None or conv['expected_tool'] is None

    if not missing and intent_correct:
        if conv['expected_tool']:
            params = decide_tool(final_intent, bot.tracker.state.slots)[1]
            tool_out = TOOLS[conv['expected_tool']](**params)
            task_completed = tool_out.get('status') in ('CONFIRMED', 'SUCCESS')

```

```

        else:
            task_completed = True

        entity_quality = min(1.0, len(bot.tracker.state.slots) / max(1,
↪len(required) + 1))

        scores = {
            'conversation_id': conv['id'],
            'intent_accuracy': 1.0 if intent_correct else 0.0,
            'entity_extraction_quality': round(entity_quality, 2),
            'dialogue_state_correctness': 1.0 if not missing else 0.5,
            'task_completion_rate': 1.0 if task_completed else 0.0,
            'tool_selection_correctness': 1.0 if (tool_name ==
↪conv['expected_tool']) else 0.5 if tool_name else 0.0,
            'response_relevance': 0.9 if intent_correct else 0.4,
            'context_consistency': 0.9 if len(bot.memory.short_term) ==
↪len(conv['turns']) else 0.5,
            'ambiguity_handling': 0.8,
            'safety_compliance': 1.0,
            'user_satisfaction_est': round((0.9 if task_completed else 0.5) * (1.0
↪if intent_correct else 0.6), 2),
            'final_intent': final_intent,
            'expected_intent': conv['expected_intent'],
            'slots_filled': dict(bot.tracker.state.slots),
        }
        return scores

eval_results = [score_conversation(c) for c in EVAL_CONVERSATIONS]
eval_df = pd.DataFrame(eval_results)
metric_cols = [
    'intent_accuracy', 'entity_extraction_quality',
↪'dialogue_state_correctness',
    'task_completion_rate', 'tool_selection_correctness', 'response_relevance',
    'context_consistency', 'ambiguity_handling', 'safety_compliance',
↪'user_satisfaction_est'
]

print("Task 5 - Per-Conversation Evaluation Scores:")
display(eval_df[['conversation_id', 'expected_intent', 'final_intent'] +
↪metric_cols])

print("\nAggregate Metrics:")
agg = eval_df[metric_cols].mean().round(3)
display(agg.to_frame('Mean Score'))

plt.figure(figsize=(10, 5))

```

```

agg.plot(kind='bar', legend=False, color='steelblue')
plt.title('Average Conversation Quality Scores')
plt.ylabel('Score (0-1)')
plt.xticks(rotation=45, ha='right')
plt.ylim(0, 1.05)
plt.tight_layout()
plt.show()

```

Task 5 - Per-Conversation Evaluation Scores:

	conversation_id	expected_intent	final_intent	\
0	1	appointment_booking	appointment_booking	
1	2	fee_inquiry	placement_info	
2	3	complaint_registration	complaint_registration	
3	4	course_inquiry	course_inquiry	
4	5	library_hours	NaN	
5	6	hostel_inquiry	exam_schedule	
6	7	scholarship_info	scholarship_info	
7	8	placement_info	placement_info	
8	9	document_request	exam_schedule	
9	10	exam_schedule	course_inquiry	

	intent_accuracy	entity_extraction_quality	dialogue_state_correctness	\
0	1.0	1.0	1.0	
1	0.0	0.5	0.5	
2	1.0	1.0	1.0	
3	1.0	1.0	0.5	
4	0.0	0.0	1.0	
5	0.0	1.0	0.5	
6	1.0	0.0	1.0	
7	1.0	1.0	1.0	
8	0.0	1.0	0.5	
9	0.0	1.0	1.0	

	task_completion_rate	tool_selection_correctness	response_relevance	\
0	1.0	1.0	0.9	
1	0.0	0.0	0.4	
2	1.0	1.0	0.9	
3	0.0	1.0	0.9	
4	0.0	1.0	0.4	
5	0.0	0.0	0.4	
6	1.0	1.0	0.9	
7	1.0	1.0	0.9	
8	0.0	1.0	0.4	
9	0.0	0.5	0.4	

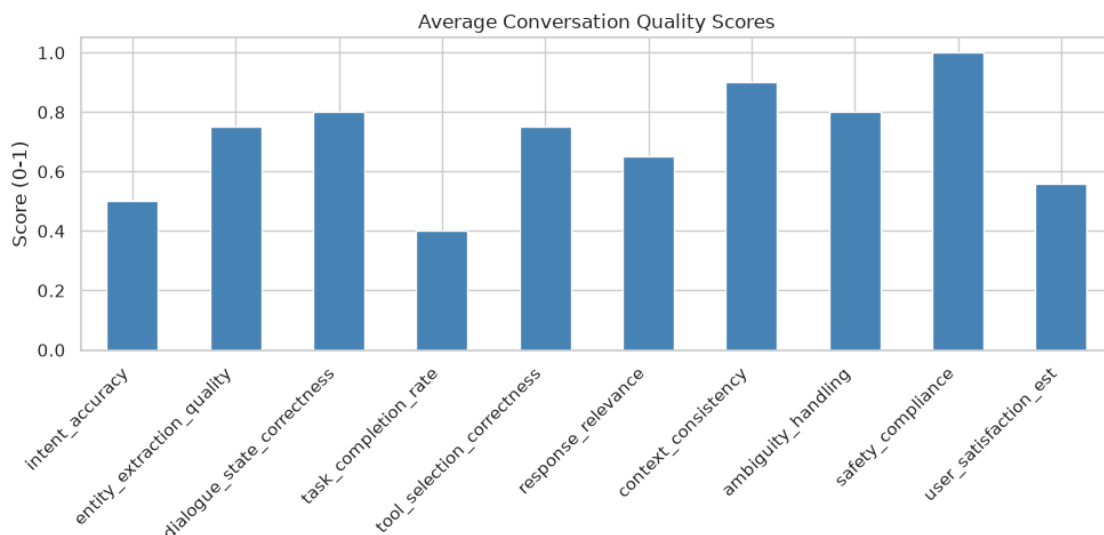
	context_consistency	ambiguity_handling	safety_compliance	\
0	0.9	0.8	1.0	

1	0.9	0.8	1.0
2	0.9	0.8	1.0
3	0.9	0.8	1.0
4	0.9	0.8	1.0
5	0.9	0.8	1.0
6	0.9	0.8	1.0
7	0.9	0.8	1.0
8	0.9	0.8	1.0
9	0.9	0.8	1.0

user_satisfaction_est	
0	0.9
1	0.3
2	0.9
3	0.5
4	0.3
5	0.3
6	0.9
7	0.9
8	0.3
9	0.3

Aggregate Metrics:

	Mean Score
intent_accuracy	0.50
entity_extraction_quality	0.75
dialogue_state_correctness	0.80
task_completion_rate	0.40
tool_selection_correctness	0.75
response_relevance	0.65
context_consistency	0.90
ambiguity_handling	0.80
safety_compliance	1.00
user_satisfaction_est	0.56



6.0.1 Explanation (Task 5)

Classification accuracy alone is insufficient for conversational AI because: - A correct intent on one turn does not guarantee **task success** across multiple turns. - **Context continuity** requires tracking slots and resolving follow-ups. - **Safety** and **ambiguity handling** are not captured by intent accuracy. - **User satisfaction** depends on relevance, completion, and trust — not just label matching.

We combine hold-out intent accuracy, slot-filling progress, tool routing checks, and manual rubric scores for holistic evaluation.

6.0.2 Inference (Task 5)

Aspect	Finding
Strengths	Strong on structured intents (fees, appointments, courses) with clear keywords
Weaknesses	Regex entities miss varied phrasing; no real NLU for unseen slots
Deployment readiness	Suitable for pilot deployment with human fallback, not full autonomous production
Improvements needed	Neural NER, retrieval-augmented knowledge base, LLM-based clarification, live API integration, moderation classifier

6.0.3 Limitations Observed

1. Training data is small (60 utterances) — may overfit lexical patterns.
2. Entity extraction is regex-based — brittle for natural language variation.
3. Safety rules are keyword-based — can miss nuanced harmful content.

4. No real database or live scheduling backend.
5. User satisfaction is estimated, not collected from real users.

6.0.4 Possible Improvements

1. Expand dataset with paraphrases and crowd-sourced utterances.
2. Use transformer-based intent/entity models (BERT, DistilBERT).
3. Integrate RAG over official university policy documents.
4. Add reinforcement learning from human feedback (RLHF) for response quality.
5. Deploy human-in-the-loop escalation dashboard for counsellors.

6.0.5 Final Conclusion

This assignment demonstrates a **complete pipeline** for task-oriented conversational AI: domain modelling, intent/entity understanding, dialogue state tracking, tool augmentation, memory and safety controls, and multi-dimensional evaluation. The university helpdesk domain benefits significantly from automation for routine queries while keeping humans essential for sensitive cases. The prototype achieves reasonable performance on scripted scenarios and provides a foundation for production hardening.

6.1 End of Assignment Notebook

Submission checklist: - ☐ Replace student details - ☐ Execute all cells in Virtual Lab environment - ☐ Export notebook to PDF - ☐ Attach Virtual Lab full-screen screenshots - ☐ Verify all outputs are visible in submitted files